İSTANBUL OKAN ÜNİVERSİTESİ · MESLEK YÜKSEKOKULU · 2026–2027 GÜZ

MBBO205 Bulut Bilişim ve Yapay Zekâ

Hafta 2 Ders Notu: Linux sistem yönetimi

İşletim sistemi, süreçler, bellek, izinler, paketler, servisler ve SSH

Dr. Öğr. Üyesi Hasan Oğuz

Bu haftanın hedefleri. Hafta sonunda öğrenci (i) çekirdeğin süreç, bellek, dosya ve aygıt yönetimindeki rolünü açıklar; (ii) süreçleri izler ve sinyallerle yönetir, bir modelin belleğe sığıp sığmayacağını kaba hesapla kestirir; (iii) kullanıcı, grup ve izinleri en az yetki ilkesine göre düzenler; (iv) apt ile paket, systemd ile servis yönetir; (v) SSH anahtarıyla uzak sunucuya bağlanıp tmux içinde kalıcı oturum açar.

İlgili kazanımlar: DÖK1, DÖK2, DÖK7 **Süre:** 1 saat teori + 1 saat uygulama

İçindekiler

1 İşletim sistemi YZ'yi nasıl taşır? 2

1.1 Çekirdeğin dört görevi . . . 2

1.2 Katmanlar ve sistem çağrısı . . . 2

2 Süreçler 3

2.1 Temel kavramlar . . . 3

2.2 Sinyaller . . . 3

3 Bellek 3

3.1 Sanal bellek, swap ve OOM killer . . . 3

3.2 Bir model belleğe sığar mı? . . . 4

4 Kullanıcılar ve izinler 4

4.1 Üç sınıf, üç bit . . . 4

4.2 En az yetki ilkesi . . . 5

5 Paketler ve servisler 5

5.1 Kim neyi kurar? . . . 5

5.2 systemd . . . 5

6 SSH ve anahtar tabanlı kimlik doğrulama 6

7 Uygulama 6

7.1 Kullanıcı ve grup . . . 6

7.2 İzinler ve paylaşımlı dizin 6
7.3 Paketler ve süreçler 6
7.4 tmux 7
7.5 SSH sunucusu ve anahtar 7

8 Laboratuvar görevi **7**
8.1 Ana görev: uzak servis durum raporu (yaklaşık 20 dakika) 7
8.2 Bozuk sistem görevi: anahtar doğru, giriş reddediliyor (yaklaşık 15 dakika) 8
8.3 Teslim listesi 8

9 Kendinizi sınayın **8**

1 İşletim sistemi YZ'yi nasıl taşır?

1.1 Çekirdeğin dört görevi

İşletim sisteminin çekirdeği (Linux), donanımı programlardan yalıtır ve dört temel görev üstlenir:

1. **Süreç yönetimi:** hangi programın ne zaman, hangi işlemci çekirdeğinde çalışacağına karar verir (zamanlayıcı).
2. **Bellek yönetimi:** her sürece yalıtılmış bir sanal adres alanı verir ve bunu fiziksel RAM'e eşler.
3. **Dosya sistemi:** veriyi diskte adlandırır, saklar ve izinleri denetler.
4. **Aygıt ve ağ:** GPU, disk ve ağ kartını sürücüler aracılığıyla programlara sunar.

Bir PyTorch eğitimi bu dört görevin hepsine aynı anda ihtiyaç duyar: bir süreç olarak çalışır, RAM ve GPU belleği (VRAM) ister, veri dosyalarını okur ve GPU'ya sürücü üzerinden erişir.

1.2 Katmanlar ve sistem çağrısı

Katman	Örnek
Uygulama	`train.py`, Jupyter
Kütüphane	PyTorch, NumPy, glibc, CUDA çalışma zamanı
Sistem çağrısı	`open`, `read`, `mmap`, `fork`, `execve`
Çekirdek	Linux: zamanlayıcı, bellek yöneticisi, sürücüler
Donanım	CPU, RAM, GPU, disk, ağ kartı

Kullanıcı alanındaki programlar çekirdeğe yalnızca **sistem çağrıları** aracılığıyla erişir. Bu sınır güvenliğin temelidir: bir programın hatası diğer programları veya çekirdeği doğrudan bozamaz.

2 Süreçler

2.1 Temel kavramlar

Çalışan her program bir **süreçtir** (process). Her sürecin benzersiz bir numarası (**PID**) vardır. Kabuk bir komutu çalıştırırken kendini `fork` ile kopyalar, kopya da `exec` ile yeni programı belleğe yükler; bu yüzden süreçler bir ebeveyn–çocuk ağacı oluşturur. Bir süreç çalışıyor (R), uyuyor (S), durdurulmuş (T) veya zombi (Z) durumunda olabilir. Komutun sonuna `&` eklemek süreci arka plana atar.

2.2 Sinyaller

Süreçlere kısa mesajlar (sinyaller) gönderilerek yönetilirler [1].

Sinyal	No	Nasıl gönderilir	Etkisi
SIGINT	2	`Ctrl+C`	Kibar kesme; program yakalayıp temizlik yapabilir
SIGTERM	15	`kill PID`	Sonlanma isteği; varsayılan sinyal
SIGKILL	9	`kill -9 PID`	Zorla öldürme; yakalanamaz
SIGTSTP	20	`Ctrl+Z`	Duraklatır; `fg/bg` ile sürer
SIGHUP	1	Terminal kapanınca	Oturuma bağlı süreçleri sonlandırır

Önce SIGTERM, son çare SIGKILL SIGKILL yakalanamaz; süreç temizlik yapamadan ölür ve yazmakta olduğu model kontrol noktası (checkpoint) bozulabilir. Uzun eğitim betikleri SIGTERM geldiğinde kontrol noktası kaydedecek biçimde yazılmalıdır.

3 Bellek

3.1 Sanal bellek, swap ve OOM killer

Her süreç kendini belleğin tamamının sahibi sanır; çekirdek bu **sanal adresleri** sayfalar hâlinde fiziksel RAM'e eşler [2]. RAM dolduğunda az kullanılan sayfalar diske (**swap**) taşınır; sistem çalışmaya devam eder ama çok yavaşlar. Swap da tükenirse çekirdeğin **OOM killer** bileşeni en çok bellek kullanan süreci SIGKILL ile sonlandırır. Belirti: eğitim hiçbir Python hatası vermeden yalnızca `Killed` yazarak kapanır. Teşhis: `sudo dmesg | grep -i oom`.

3.2 Bir model belleğe sığar mı?

Bir modelin yalnızca ağırlıklarını tutmak için gereken bellek, parametre sayısı N_{param} ile parametre başına bayt b'nin çarpımıdır:

$$M \approx N_{\text{param}} \times b. \tag{1}$$

Sayı biçimi	b (bayt)	7 milyar parametre
fp32	4	28 GB
fp16 / bf16	2	14 GB
int8	1	7 GB
int4	0,5	3,5 GB

Çözümlü örnek. 8 GB RAM'li bir dizüstünde 7 milyar parametreli bir dil modeli çalışır mı? fp16'da $7 \times 10^9 \times 2\,\text{B} = 14\,\text{GB}$ gerekir: sığmaz. int4'e nicemlenmiş sürüm $7 \times 10^9 \times 0{,}5\,\text{B} = 3{,}5$ GB ağırlık ister; işletim sistemi ve bağlam önbelleği için pay bırakılırsa çalışabilir. Eğitim çok daha pahalıdır: Adam optimizasyonuyla her parametre için ağırlık, gradyan ve iki moment tutulur; bu, parametre başına yaklaşık 16 bayt demektir [3]. Aynı modeli eğitmek için yalnızca bu durumlar $7 \times 10^9 \times 16 \approx 112$ GB gerektirir; etkinleştirmeler (activations) bunun üstüne eklenir. Formül bu nedenle bir alt sınırdır.

4 Kullanıcılar ve izinler

4.1 Üç sınıf, üç bit

Her dosyanın bir sahibi (**u**), bir grubu (**g**) ve bir de "diğerleri" (**o**) vardır. Her sınıf için okuma (**r**), yazma (**w**) ve çalıştırma (**x**) izni ayrı ayrı tanımlanır. Her sınıfın izni tek bir sekizlik rakamla yazılır:

$$m = 4r + 2w + x, \qquad r, w, x \in \{0, 1\}. \tag{2}$$

Sayı	Sembol	Tipik kullanım
755	`rwxr-xr-x`	Betikler ve dizinler
644	`rw-r--r--`	Normal dosyalar
700	`rwx------`	`~/.ssh` dizini
600	`rw-------`	Özel anahtar, `authorized_keys`
2770	`rwxrws---`	Grupla paylaşılan veri dizini (setgid)

Çözümlü örnek. `chmod 750 rapor.sh` ne yapar? $7 = 4 + 2 + 1$: sahip okur, yazar, çalıştırır. $5 = 4 + 1$: grup okur ve çalıştırır. 0: diğerleri hiçbir şey yapamaz. Sonuç `rwxr-x---`. *Dizinlerde* `x` biti "dizine girme" iznidir: `r` olup `x` yoksa `ls` dosya adlarını listeler ama `cd` başarısız olur.

Başa eklenen dördüncü rakam özel bitleri taşır: **2** (setgid) bir dizinde oluşturulan yeni dosyaların dizinin grubunu miras almasını sağlar; ekip çalışması için idealdir.

4.2 En az yetki ilkesi

`root` kullanıcısı (UID 0) tüm izin denetimlerini atlar. Bu yüzden günlük çalışma normal kullanıcıyla yapılır; yönetim işleri tek tek `sudo` ile yürütülür ve her kullanım günlüğe yazılır. Paylaşım için dizini `777` yapmak yerine bir grup oluşturup dizine `2770` izni vermek doğrudur. Veritabanı ve web servisleri kendi kısıtlı kullanıcılarıyla çalışır; biri ele geçirilse bile saldırganın yetkisi sınırlı kalır.

5 Paketler ve servisler

5.1 Kim neyi kurar?

Araç	Neyi yönetir	Örnek
`apt`	Sistem paketleri (Ubuntu depoları)	`sudo apt install htop`
`pip`	Python paketleri (PyPI)	`pip install numpy`
`uv`	Hızlı Python ortamı ve paket yöneticisi	`uv pip install pandas`
`conda`	Python ve derlenmiş kütüphaneler	`conda install pytorch`
`docker`	Tüm ortam bir imajda (hafta 8)	`docker pull postgres`

Kural: sistem araçları `apt` ile, proje kütüphaneleri bir sanal ortama kurulur. Ubuntu 24.04, sistem Python'una `pip` ile paket kurulmasını PEP 668 ile engeller (hafta 5).

5.2 systemd

Arka planda sürekli çalışan programlara servis (daemon) denir: SSH sunucusu, veritabanı, model API'si, Jupyter. Ubuntu'da bunları **systemd** yönetir: açılışta başlatır, çökerse yeniden başlatır, günlüklerini **journald** içinde toplar. Bir servisin tarifi bir *unit* dosyasıdır [4]. 14. haftada model servisimizi aşağıdakine benzer bir dosyayla çalıştıracağız:

```
# /etc/systemd/system/mbbo205-demo.service
[Unit]
Description =MBBO205 ornek servis
After =network.target

[Service]
User =ogrenci
WorkingDirectory =/home/ogrenci/mbbo205
ExecStart =/usr/bin/python3 -m http.server 8000
Restart =on-failure

[Install]
WantedBy =multi-user.target
```

`sudo systemctl daemon-reload && sudo systemctl enable --now mbbo205-demo` ile etkinleştirilir; `journalctl -u mbbo205-demo` ile günlüğü okunur.

6 SSH ve anahtar tabanlı kimlik doğrulama

Parola yerine bir **anahtar çifti** kullanılır [5]:

1. `ssh-keygen` bir özel (`id_ed25519`) ve bir açık (`id_ed25519.pub`) anahtar üretir.
2. Açık anahtar sunucudaki `~/.ssh/authorized_keys` dosyasına eklenir.
3. Bağlanırken sunucu bir meydan okuma gönderir; istemci onu özel anahtarla imzalar.
4. Sunucu imzayı açık anahtarla doğrular; parola ağda hiç dolaşmaz.

Özel anahtar bilgisayarınızdan asla çıkmaz, paylaşılmaz ve Git deposuna girmez. SSH sunucusu (`sshd`) güvenlik için dosya izinlerini denetler: `StrictModes` açıkken (varsayılan) başkalarının yazabildiği bir `.ssh` dizinini güvensiz sayar ve anahtarı reddeder [6].

7 Uygulama

7.1 Kullanıcı ve grup

```
whoami; id; groups
sudo adduser deneme
sudo groupadd ai-lab
sudo usermod -aG ai-lab $USER      # -a unutulursa diğer gruplardan çıkarılırsınız
sudo usermod -aG ai-lab deneme
getent group ai-lab
newgrp ai-lab                      # üyelik yeni oturumda geçerli olur
```

7.2 İzinler ve paylaşımlı dizin

```
ls -l ~/mbbo205/hafta01
chmod 644 env_report.sh; ls -l env_report.sh
chmod u+x env_report.sh
sudo mkdir -p /srv/veri
sudo chgrp ai-lab /srv/veri
sudo chmod 2770 /srv/veri
ls -ld /srv/veri                   # drwxrws--- beklenir; s = setgid
```

7.3 Paketler ve süreçler

```
sudo apt update                    # yalnızca paket listesini yeniler
apt show htop
sudo apt install -y htop tmux tree
dpkg -L tmux | grep bin
sleep 600 &                        # arka planda uzun bir iş
jobs; pgrep -a sleep
htop                               # F6 sırala, F9 sinyal gönder, q çık
kill %1                            # SIGTERM
sudo dmesg | grep -i oom           # OOM olayları var mı?
```

7.4 tmux

Terminal kapandığında SIGHUP oturumdaki süreçleri sonlandırır. tmux oturumu terminalden bağımsız yaşar; SSH bağlantısı kopsa bile içindeki eğitim devam eder.

```
tmux new -s egitim
for i in $(seq 300); do date; sleep 1; done
# Ctrl+b d  -> oturumdan ayrıl (detach)
tmux ls
tmux attach -t egitim
```

7.5 SSH sunucusu ve anahtar

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
systemctl status ssh
sudo ss -tlnp | grep :22
journalctl -u ssh -n 20 --no-pager
ssh-keygen -t ed25519 -C "ogrenci@okan"
ssh-copy-id ogrenci@localhost
ssh ogrenci@localhost
```

WSL kullananlar bu hafta kendi makinelerine `localhost` üzerinden bağlanır. Güncel Ubuntu WSL imajlarında systemd açıktır; kapalıysa `/etc/wsl.conf` dosyasına `[boot]` bölümü altında `systemd =true` yazılır ve PowerShell'de `wsl --shutdown` çalıştırılır. Kısa ad için `~/.ssh/config`:

```
Host lab
    HostName localhost
    User ogrenci
```

8 Laboratuvar görevi

8.1 Ana görev: uzak servis durum raporu (yaklaşık 20 dakika)

```
cat > rapor.sh << 'EOF'
hostname; uptime
systemctl is-active ssh
free -h | grep Mem
df -h / | tail -1
ps aux --sort =-%mem | head -5
EOF
ssh lab 'bash -s' < rapor.sh > servis_raporu.txt
cat servis_raporu.txt
```

`bash -s` betiği standart girdiden okur; betik sunucuya kopyalanmadan orada çalışır, çıktısı yerel dosyaya yazılır.

8.2 Bozuk sistem görevi: anahtar doğru, giriş reddediliyor (yaklaşık 15 dakika)

Bir arkadaşınız "kolaylık olsun" diye şunu çalıştırmış:

```
$ chmod -R 777 ~/.ssh
$ ssh lab
ogrenci@localhost's password:
$ journalctl -u ssh -n 5 --no-pager
... Authentication refused: bad ownership or modes for directory /home/ogrenci/.ssh
```

Görev: belirtiyi kaydedin; en az iki hipotez yazın; `ssh -v lab` ve sunucu günlüğüyle test edin; izinleri onarın; `KOK_NEDEN_RAPORU.md` şablonunu doldurun.

İpucu: fazla izin de bir hatadır. `ls -ld ~/.ssh` ve `ls -l ~/.ssh` çıktısını yukarıdaki izin tablosuyla karşılaştırın.

8.3 Teslim listesi

- `servis_raporu.txt`
- `KOK_NEDEN_RAPORU.md` (SSH izinleri)
- `/srv/veri` için `ls -ld` çıktısı
- `AI_USAGE.md` (üretken YZ kullandıysanız)

Son teslim: [tarih, saat] Teslim yeri: [LMS / e-posta]

9 Kendinizi sınayın

1. Bir Python betiği neden doğrudan diske yazamaz da `open` ve `write` sistem çağrılarını kullanır?
2. `kill PID` ile `kill -9 PID` arasındaki fark nedir? Hangisi bir eğitim betiği için daha güvenlidir, neden?
3. 13 milyar parametreli bir model int8 biçiminde en az ne kadar bellek ister? 16 GB'lık bir GPU'ya sığar mı?
4. `chmod 640 notlar.txt` komutundan sonra `ls -l` çıktısının izin bölümü ne olur?
5. Bir dizinde `r` izni olup `x` izni yoksa hangi komutlar çalışır, hangileri çalışmaz?
6. `usermod -G ai-lab ali` komutu `-a` olmadan çalıştırılırsa ne olur?
7. SSH bağlantısı koptuğunda `nohup` ile tmux arasındaki fark nedir?
8. `sshd` neden `777` izinli bir `.ssh` dizinindeki anahtarı reddeder? Bu davranışın koruduğu saldırı senaryosunu tarif edin.

Hafta 3 için hazırlık

SSH kurulumunu tamamlayın; GNU Awk kılavuzunun giriş bölümünü okuyun [7]; MIT Missing Semester "Shell Tools and Scripting" dersini izleyin [8]. Gelecek hafta: algoritma karmaşıklığı, Bash betikleme, `grep`, `sed`, `awk`, `jq` ve `cron`.

Kaynaklar

[1] Linux man-pages, *signal*(7). `https://man7.org/linux/man-pages/man7/signal.7.html`

[2] The Linux Kernel documentation, "Concepts overview" (memory management). `https://docs.kernel.org/admin-guide/mm/concepts.html`

[3] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," SC20 (2020). `https://arxiv.org/abs/1910.02054`

[4] freedesktop.org, *systemd.service*(5). `https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html`

[5] OpenBSD manual pages, *ssh-keygen*(1). `https://man.openbsd.org/ssh-keygen`

[6] OpenBSD manual pages, *sshd_config*(5), StrictModes. `https://man.openbsd.org/sshd_config`

[7] Free Software Foundation, *The GNU Awk User's Guide*. `https://www.gnu.org/software/gawk/manual/gawk.html`

[8] MIT, *The Missing Semester*, "Shell Tools and Scripting." `https://missing.csail.mit.edu/2020/shell-tools/`

[9] tmux wiki, "Getting Started." `https://github.com/tmux/tmux/wiki/Getting-Started`